

Spatiotemporal Sketch Disaggregation: Streaming Analytics with Heterogeneous Resources

Jonatan Langlet

Electrical Engineering and Computer Science
KTH Royal Institute of Technology
Stockholm, Sweden
jlanglet@kth.se

Peiqing Chen

Department of Computer Science
University of Maryland, College Park
College Park, MD, USA
pqchen99@umd.edu

Michael Mitzenmacher

Engineering and Applied Sciences
Harvard University
Allston, MA, USA
michaelm@eecs.harvard.edu

Zaoxing Liu

Department of Computer Science
University of Maryland, College Park
College Park, MD, USA
zaoxing@cs.umd.edu

Ran Ben Basat

Computer Science department
University College London
London, United Kingdom
r.benbasat@cs.ucl.ac.uk

Gianni Antichi

Dept. of Electronics, Information and Bioengineering
Politecnico di Milano
Milano, Italy
gianni.antichi@polimi.it

Abstract—Streaming analytics are essential in a large range of applications, including databases, networking, and machine learning. To optimize performance, practitioners are increasingly offloading such analytics to network nodes such as switches. However, resources such as fast SRAM memory available at switches are limited, not uniform, and may serve other functionalities as well (e.g., firewall). Moreover, resource availability changes over time due to the dynamic demands of in-network applications.

In this paper, we propose a new approach to disaggregating data structures, leveraging any residual resources available at network nodes. We focus on sketches, which are fundamental for summarizing data for streaming analytics while providing beneficial space-accuracy tradeoffs. Our idea is to break sketches into multiple ‘fragments’ that are placed at different network nodes. The fragments cover different time periods and vary in size, and are combined to form a network-wide view of the underlying traffic. We apply our solution to three popular sketches (namely, Count Sketch, Count-Min Sketch, and UnivMon) and demonstrate that we can achieve approximately a 75% memory size reduction for the same error for many queries, or a near order-of-magnitude error reduction if memory is kept unchanged. Further, we demonstrate real-world feasibility through a hardware pipeline for high-speed commodity switches.

Index Terms—Distributed data structures, Network monitoring, Computations on discrete structures, Pipeline implementation

I. INTRODUCTION

Streaming analytics structures are central to both databases [33], [57], [79] and networking [36], [77], [78], enabling tasks such as identifying the most frequently accessed records or detecting heavy flows in real time [6]. These structures make it possible to analyze massive streams of events with modest resources, which is essential for real-time monitoring and decision-making at scale. To improve performance and reduce costs, many of these functions are increasingly offloaded to programmable network switches [13], [64], where analytics can be performed directly on packets in flight.

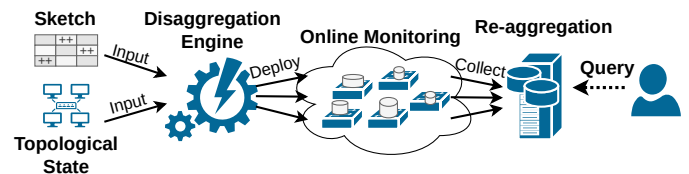


Fig. 1: Overview of Sketch Disaggregation.

However, switches offer only on the order of 10MB of line-rate memory and have limited computational capabilities [14], [15], [47], [63]. Such restrictions are necessary to sustain terabits per second of throughput. Moreover, these scarce resources must be shared among diverse applications, including streaming analytics [8], [58], [69], security [59], [75], [76], machine learning aggregation [49], [54], [67], database acceleration [52], [57], [70], and various network functionalities [39], [56], [60], [65]. Because different programmable switches run different workloads, and requirements can change dynamically with traffic or queries, the available memory varies greatly and changes constantly. Pre-allocating large, fixed-size analytics structures thus risks either wasting valuable resources when underutilized, or failing to meet accuracy demands when the network is heavily loaded. This motivates designs that can opportunistically harness whatever residual memory happens to be free at each switch at any given time.

In this paper, we propose leveraging switches’ *residual resources* as ‘fragments’ of a disaggregated data structure. We focus on sketches, which are fundamental and typically aggregated building blocks for streaming analytics. Our method splits a sketch into multiple, dynamically sized fragments that adapt in real time and are centrally combined to answer queries as if they were a single structure (Fig. 1).

Most prior work on sketches has aimed to minimize memory use in aggregated deployments by optimizing the size-accuracy tradeoff [18], [20], [24], [58], [62]. This approach is limited by the resources of a single node and misses the opportunity to exploit residual memory across the network.

Application	Examples	SRAM
Basic Packet Processing	switch.p4 [61]	30%
Security	Ripple [76], Jaen [59], Bedrock [75]	+10-50%
Machine Learning	SwitchML [67], ATP [49], THC [54]	+10-40%
Storage/Database	DistCache [57], NETACCEL [52], Cheetah [70]	+20-30%
Networking	SilkRoad [60], HPCC [56], SwRL [39], Sailfish [65]	+5-40%

TABLE I: The on-switch memory cost of network functions.

Our key observation is that packets typically traverse several nodes (e.g., switches). Each node can contribute a fragment, and combining them allows us to harness heterogeneous memory capacities along a path to improve accuracy, even when flows follow different routes. Although we focus on network analytics, the same principles apply to memory-constrained distributed databases, where key searches span multiple nodes and can leverage intermediate resources such as per-node CPU caches or even in-network accelerators [1], [73].

Although conceptually simple, designing such disaggregated sketches is challenging because: (1) fragments differ in available resources, leading to unequal accuracy, (2) traffic volumes vary across nodes, and (3) flows traverse paths of different lengths, resulting in different numbers of contributing fragments. In addition, switches have severely limited computational capabilities: even basic operations such as multiplication or division are typically unsupported [11], and only minimal per-packet processing is feasible at line rate [63]. Any practical design must therefore be lightweight, self-tuning, and compatible with the constraints of high-speed data planes.

In our solution, time is divided into consecutive ‘epochs’ that define the granularity for user queries. Our main innovation is to divide each fragment’s epoch into smaller subepochs, after which the fragment is collected. Each flow is measured in a single subepoch per epoch, enabling dynamic use of hardware resources through flow-level temporal sampling. This design trades memory for reporting frequency: when memory is constrained, more subepochs keep errors within bounds; when memory is abundant, fewer subepochs reduce collection overhead.

To answer queries, we combine fragments that measure a flow along its path. Estimates from shorter subepochs are scaled for comparability across fragments. This relies on the assumption that flow rates remain stable within a short epoch, allowing accurate estimates to be derived from a single subepoch.

We realize this technique in *DiSketch*: a system that efficiently disaggregates sketches to leverage residual resources while optimizing accuracy. We demonstrate generality by applying it to three widely used sketches: Count Sketch [18], Count-Min Sketch [24], and UnivMon [58], and we compare its performance against traditional aggregated deployments and DISCO [16]. We also analyze hardware costs relative to Distributed Sketch [35], showing that *DiSketches* remain lightweight. Thorough experiments with real-world traffic show that *DiSketch* is highly effective, matching DISCO’s accuracy with just 25% of the memory, or reducing error by nearly an order of magnitude under the same memory budget.

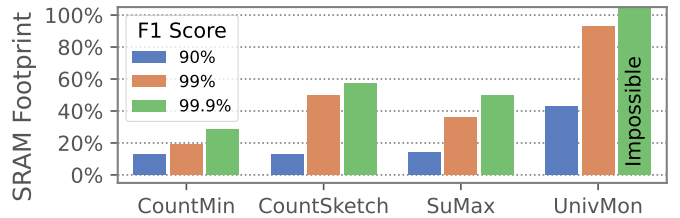


Fig. 2: On-switch memory cost to achieve an accuracy target while monitoring 30s of real-world backbone traffic [17].

II. MOTIVATION

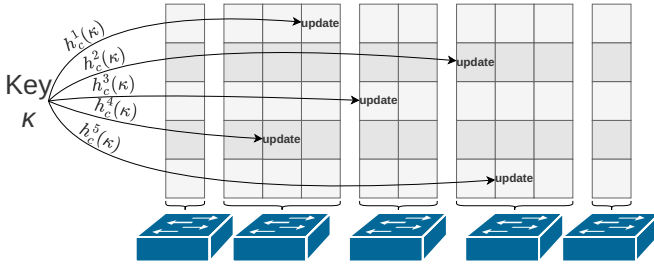
Recent advancements in streaming analytics have led to the development of compact sketch-based solutions, as described in several works [28], [40]–[42], [58], [62], [74], [80]. These studies have demonstrated the feasibility of implementing these structures within the constraints of modern switches, achieving minimal estimation errors [41], [42], [62], [80]. However, sketches remain memory-intensive, and their accuracy directly depends on the resources allocated to them. As we show later, deploying sketches on switches alongside other functionality competing for limited memory can result in significant accuracy degradation.

Our survey of recent literature on in-network computing, covering applications from security to machine learning acceleration and network functions, shows a significant demand for switch resources. For example, basic packet processing, such as L2/L3 forwarding, may consume up to 30% of a switch’s memory (Table I), and adding further functionalities reduces the memory available for sketches.

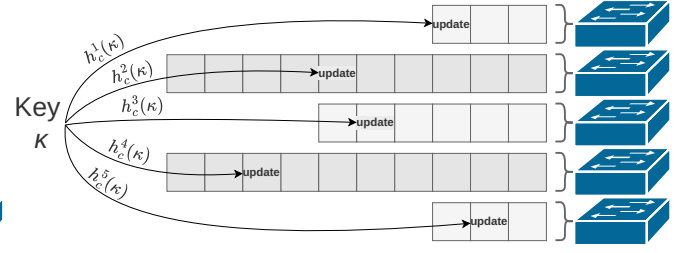
To quantify the impact of this resource competition, we analyzed the SRAM requirements of both established sketches (i.e., Count Sketch [18], Count-Min-Sketch [24]) and more recent ones (i.e., UnivMon [58], SuMax [80]) for heavy-hitter detection over a 30-second window (similar to previous works [58], [74], [77]), using real-world Internet backbone traces [17]. Our findings, shown in Fig. 2, reveal that achieving a 99% F1 score requires between 20% and nearly 90% of a programmable Tofino switch’s SRAM [63], underscoring the difficulty of maintaining high monitoring accuracy when sketches share resources with other functions. A high monitoring accuracy is essential as the base of responsive diagnosis [55], [68], yet even 99% accuracy already leads to a significant amount of incorrect flow classification. Achieving higher accuracy requires even more memory and can, in some cases, exceed the switch’s capacity even in isolation.

Moreover, even if a sketch is not co-located with other in-network functions, the amount of traffic observed by each switch can differ significantly, with volumes varying by orders of magnitude even among switches with the same logical role (e.g., edge switches in a datacenter) [9], [27], [66]. As a consequence, deploying monitoring on a cut in the network topology (e.g., all edge switches [37]) results in varying accuracy levels even when all switches have the same memory allocated.

Acknowledging these challenges, disaggregating sketches across multiple switches emerges as a promising solution for utilizing network-wide resources.



(a) Per-column disaggregation. Fragments host full-depth sketches.



(b) Per-row disaggregation. Fragments host one row each.

Fig. 3: Visualization of the sketch disaggregation directions. Note how per-row always uses exactly one counter per fragment.

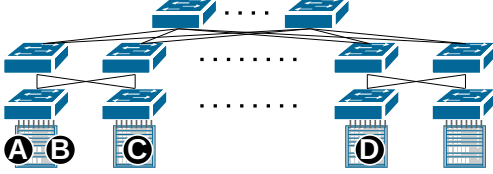


Fig. 4: A Fat-Tree Topology. Packets traverse 1 (e.g., A-B), 3 (e.g., A-C), or 5 (e.g., A-D) switches.

III. SKETCH DISAGGREGATION

We consider disaggregating *sketches*, which, for our purposes, are viewed as matrix structures in which each cell is an identical copy of a simpler data structure, usually a counter. We further assume that when an element (e.g., a packet) is inserted into the data structure, its key (e.g., flow ID) is mapped via hashes into one or more cells in each row, and the cells are updated appropriately. In what follows, we assume that one cell is updated in each row and that this cell is chosen uniformly by a hash function. Examples of such sketches include ones for frequency estimation [18], [25], [29], [42], [80], set membership [10], [28], [30], sparse recovery [32], [43], frequency moment estimation [4], [19], [58], entropy estimation [21], [38], [58], and ℓ_p samplers [22], [23]. These sketches' cells are exported and reset periodically to prevent the buildup of stale data, which degrades estimation accuracy. The period between resets is referred to as the *epoch*.

Understanding the challenges of sketch disaggregation across multiple nodes begins with a depiction of a datacenter network's architecture. A classic Fat-Tree topology, commonly referenced in literature and employed in real-world deployments, is illustrated in Fig. 4. This topology highlights the existence of numerous paths between any two end nodes, with the number of hops varying based on the nodes' locations. For example, flows between (A) and (B) traverse a single switch, while any path between (A) and (D) contains five switches. Sketch disaggregation is then the process of dividing a sketch across network paths, where each network node hosts a fragment of the sketch. As in traditional deployments, we assume that at the end of each epoch, per-node data structures are sent for analysis to a central server called the controller. After collection and aggregation, per-path fragments can be queried together, as in traditional aggregated sketches.

When distributing sketches across multiple switches, the central design choice is how to split the sketch matrix among nodes. There are two natural approaches to disaggregating sketches: *per-column* and *per-row* disaggregation (Fig. 3).

In per-column disaggregation (Fig. 3a), each network hop hosts all sketch rows of the sketch matrix but only a subset of the columns. Keys are still mapped to one cell in each row, selected uniformly at random by a hash function. This requires all sketch nodes to know both the path length and the fragment widths as a packet traverses the network, which is costly. That is because a switch must know the indices of the columns it holds and the total number of on-path columns, since the hash functions must output a column index. Previous work on sketch disaggregation attempts to solve this through lookup tables in each fragment containing entries for every network path that traverses a switch [35], which poses a scaling issue for large networks. For example, on a k -ary Fat-Tree, each edge switch needs to store information about $k^{3 \cdot (k-1)/8}$ paths (it has $k/2$ options for each of the aggregate and core switches, $k-1$ for the aggregate switch on the way down, and another $k/2$ for the last edge switch). This means that even for moderately sized networks such as $k = 28$, we would require information for about 74 thousand paths, leaving less memory for the sketch itself and undermining the purpose of the disaggregation.

In contrast, with per-row disaggregation (Fig. 3b), each node hosts a single row in each fragment, occupying all sketch-allocated memory. Each fragment is then equivalent to an independent sketch row, so it can function in isolation, and no lookup table is needed as in per-column disaggregation. Note that, unlike standard sketches, differences in row sizes due to memory lead to an “irregularly” shaped matrix, which makes it harder to aggregate the results into a single accurate estimate.

To illustrate fundamental overheads, we implemented state-of-the-art per-row (DISCO [16]) and per-column (Distributed Sketch [35], without a lookup table) disaggregated sketches on a programmable Tofino switch [63]. Fig. 5 presents these overheads by comparing the total resources required along a path with those of a traditional Count Sketch. While per-row is more efficient than per-column disaggregated sketches, producing accurate estimates is particularly challenging when switches differ significantly in memory and traffic volumes.

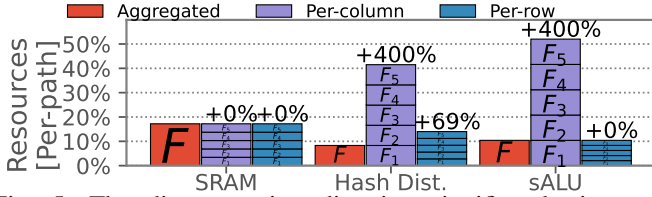


Fig. 5: The disaggregation direction significantly impacts costs. Shown here are full-path hardware costs.

The main challenges of per-row disaggregation are:

Challenge 1: Nodes across the network can have varying resources due to distinct in-network functions deployed at different switches as well as diverse hardware. Some functions, such as security mechanisms, may be more suitably deployed at switches near endpoints [67], [70], [75], while others fit better within the network core [65]. This diversity leads to a heterogeneous use of memory, ruling out per-column disaggregation due to the high computational overhead and substantial memory requirements for stateful per-flow counter allocation. Per-row disaggregation in highly heterogeneous deployments can suffer accuracy degradation, where small fragments introduce significant errors to the composite sketch. Alternatively, these fragments become essentially negligible when assigned an importance inversely proportional to their relatively high error.

Challenge 2: Traffic volume can vary significantly across nodes, stemming from the design of datacenter networks and their traffic patterns. For example, the Fat-Tree topology facilitates massive multipath routing [2], yet despite load balancing efforts, imbalances persist [3], [34], [45], [46], [72]. Studies have shown that much traffic remains local, with only a fraction traversing the entire datacenter [66]. Consequently, switches near end-hosts experience higher traffic volumes than those at the core, affecting the accuracy of sketch fragments under heavy loads.

Challenge 3: Flows traverse paths of varying lengths, from rack-local traffic observed at a single switch to multi-hop flows spanning the network. As a result, some flows benefit from multiple fragments, while single-hop flows rely on just one fragment, effectively a one-row sketch that is often inaccurate.

We demonstrate this effect with DISCO, a per-row disaggregated Count Sketch, in a simulated Fat-Tree topology using the same experimental parameters as in Sec. VI-A. We deploy equal-sized per-row fragments on every switch and replay 30 seconds of real-world CAIDA traffic [17] across the network. Flows are queried individually, and their estimation accuracies are recorded. To account for hash collisions, we repeat the experiment several times with different hash functions and report per-path-length accuracy in Fig. 6: single-hop flows are significantly less accurate than 3- and 5-hop flows, with 5-hop flows performing best.

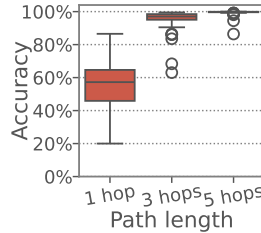


Fig. 6: Accuracy while sketching under varying flow path lengths.

In the next section, we introduce a general technique for deploying per-row disaggregated sketches to achieve accurate estimates under tight and heterogeneous memory.

IV. SPATIOTEMPORAL DISAGGREGATION

In heterogeneous networks, fragments that are small or heavily loaded yield higher estimation noise. In some cases, it can even be better to ignore such fragments and rely on larger on-path ones, which wastes residual resources and indicates a deeper fundamental inefficiency.

A natural idea is flow-level sampling: each per-switch sketch instance (fragment F) tracks only a subset of flows sized to its memory and load. Large or lightly loaded fragments admit more flows; congested ones admit fewer. This approach helps, but it risks leaving flows untracked when all on-path fragments skip them, and it also risks arbitrarily creating uneven accuracy across flows regardless of path.

We instead propose a novel *temporal* sampling technique. Each fragment operates over an epoch E that it splits into n equal subepochs $S \in \{0, \dots, n-1\}$. A key κ (e.g., flow, host, or port) is monitored in exactly one subepoch per fragment via a per-epoch map $s_E^F(\kappa)$. At the end of each subepoch, the fragment exports a record $R = (F, E, S, n, c, h)$ with counters c and hash functions h . Central queries assemble a composite sketch from the relevant records.

We start with a brief overview of our solution, followed by a more detailed description in subsequent sections. See Fig. 7 for a visual overview.

Subepoching (§IV-A): Each fragment divides its sketching epoch into n subepochs, where n is chosen per fragment. A key κ is monitored during one subepoch in each on-path fragment, so a fragment tracks approximately $1/n$ of the keys at a time. Counters are exported and reset after each subepoch to allow for central querying. We elaborate in Sec. IV-A.

Error Equalization (§IV-B): A network-wide error target is chosen to homogenize errors across fragments and facilitate network-wide querying. Fragments estimate their local errors at the end of each epoch and adjust the number of subepochs for the upcoming epoch so that all fragments aim to deliver similar error bounds. We elaborate in Sec. IV-B.

Central Queries (§IV-C): We consider queries that analyze traffic in one or several adjacent sketching epochs. These queries target a specific key (e.g., a flow, port, or host) or the aggregate network (e.g., entropy). The exact supported queries depend on the deployed sketch. At query time, collected data from relevant fragments is used to compile a virtual composite sketch that covers the queried data streams. We elaborate in Sec. IV-C.

A. Subepoching

First, we present subepoching: the foundation of our temporal sampling. For ease of exposition, we first assume that all flows follow the same path length; in Sec. IV-D, we discuss an optimization that improves accuracy when path lengths differ.

Fragments dynamically divide their epochs into a power-of-two number of subepochs ($n = 2^x$ for some $x \in \mathbb{N}$).

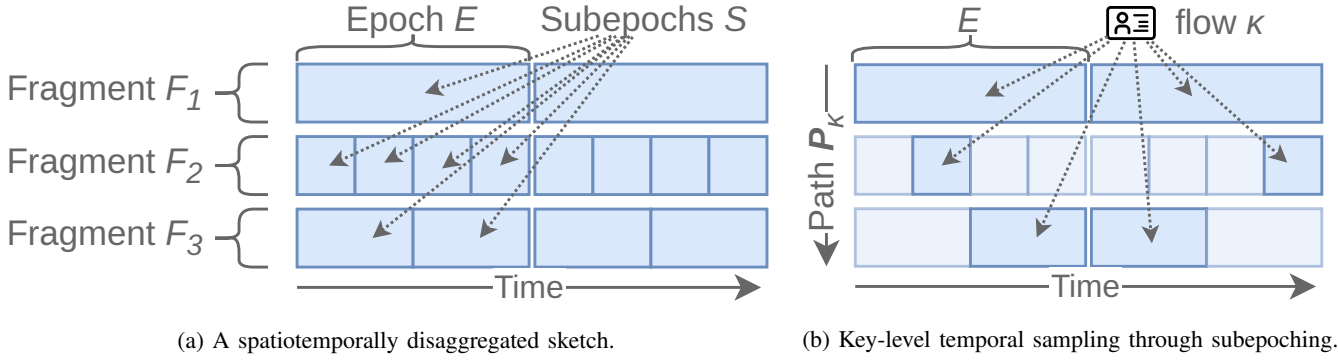


Fig. 7: Fragment F splits epoch E into n_E^F subepochs S . Key κ on path P_κ maps to one subepoch per fragment and epoch.

During epoch E , fragment F locally divides the global epoch duration into n_E^F equal-sized subepochs. We hereafter omit the F superscript when unambiguous.

Subepoching performs temporal key-level sampling while guaranteeing queryability within each query window. Each fragment maps κ to one subepoch per epoch via a fragment-specific hash $s_E^F : \kappa \rightarrow \{0, 1, \dots, n_E - 1\}$, so each subepoch monitors a distinct subset of keys (Fig. 7b).

A subepoch record is generated and exported at the end of each subepoch, containing all information required to centrally query the data stream (see Sec. IV-C).

At the start of a new epoch, fragments replace their hash functions to avoid persistent collisions and recompute n based on estimated performance to equalize network-wide error bounds.

B. Error Equalization

In most cases, sketches provide accuracy guarantees by analyzing a basic data structure (e.g., a sketch row in Count-Min and Count Sketch) that yields the desired precision with a constant probability (e.g., $3/4$). Merging independent repetitions of this structure (e.g., using the minimum in Count-Min or the median in Count Sketch) amplifies the success probability as desired. For simplicity, and because we find it effective, we follow the same rationale: we aim for different fragments to yield estimates with similar errors, allowing us to amplify success probability through merging.

With this in mind, we consider Count-Min and Count Sketch as examples. The standard analysis of Count-Min examines the expected noise that other keys impose on the queried key's counter. Assuming a pairwise independent hash, any other key increases the counter with probability $1/w$, so the expected noise is bounded by $\sum_{\kappa \in \mathbf{K}} f_\kappa/w$. By Markov's inequality,

$$\Pr \left[\hat{f}_\kappa - f_\kappa \geq 4 \sum_{\kappa \in \mathbf{K}} f_\kappa/w \right] \leq 1/4 \quad (1)$$

where f_κ is the frequency of $\kappa \in \mathbf{K}$ and w is the number of sketch counters.

Similarly, following the standard analysis of Count Sketch [18], [50], each fragment has a frequency estimate $\hat{f}_\kappa$ for every key $\kappa \in \mathbf{K}$ with expectation f_κ and variance at most $\sum_{\kappa \in \mathbf{K}} f_\kappa^2/w$. Using Chebyshev's inequality,

$$\Pr \left[|\hat{f}_\kappa - f_\kappa| \geq 2 \sqrt{\sum_{\kappa \in \mathbf{K}} f_\kappa^2/w} \right] \leq 1/4. \quad (2)$$

Therefore, based on the switch's available space, we size subepochs so that the noise bound roughly matches a target quantity ρ , which we loosely refer to as a fragment's probabilistic error bound (PEB). The magnitude of a fragment's estimation errors is linked to ρ . From the above, we set

$$\rho = \begin{cases} \sqrt{\sum_{\kappa \in \mathbf{K}} f_\kappa^2/w}, & \text{if CS} \\ \sum_{\kappa \in \mathbf{K}} f_\kappa/w, & \text{if CMS} \end{cases} \quad (3)$$

To roughly equalize error across fragments, we define a network-wide *target PEB* ρ_{target} , the desired PEB in all fragments' subepoch records. Each switch, knowing its space constraints, attempts to meet this target.

For our purposes, we treat sketch fragments as single-row sketches. Since sketches estimate the underlying frequency vector $\mathbf{f}$, we use the fragment's counters $c_i \in \mathbf{c}$ as an approximation and modify Equation 3 to calculate an estimated subepoch PEB $\hat{\rho} \approx \rho$:

$$\hat{\rho} = \begin{cases} \sqrt{\sum_{i=1}^w c_i^2/w}, & \text{if CS} \\ \sum_{i=1}^w c_i/w, & \text{if CMS} \end{cases} \quad (4)$$

Let $\hat{\rho}_{E,s}$ be the estimated PEB from subepoch s in epoch E . An epoch's average error bound is then estimated as

$$\overline{\hat{\rho}}_E = \frac{1}{n_E} \sum_{s=0}^{n_E-1} \hat{\rho}_{E,s}. \quad (5)$$

We aim to equalize $\overline{\hat{\rho}}_E$ across the network so that $\overline{\hat{\rho}}_E \approx \rho_{\text{target}}$ for all fragments and epochs. Recall that $n \approx \text{FlowSamplingRate}^{-1}$ within each subepoch, which leads to $\rho \propto 1/n$. We also assume that traffic patterns are relatively stable between consecutive epochs, hence $\overline{\hat{\rho}}_{E+1} \approx \overline{\hat{\rho}}_E$ for each epoch. Fragments then adjust n to approach ρ_{target} :

$$n_{E+1} = \begin{cases} 2n_E, & \text{if } \overline{\hat{\rho}}_E > 2\rho_{\text{target}} \\ \max(1, n_E/2), & \text{if } \overline{\hat{\rho}}_E < \rho_{\text{target}}/2 \\ n_E, & \text{otherwise} \end{cases} \quad (6)$$

ρ_{-1} is not defined, so we initiate the first epoch with $n_0 = 1$. We recommend computing a moving n_{E+1} as in Equation 6 instead of independently calculating it (through $n_{E+1} = 2^{\lfloor \max(0, \log_2 \overline{\hat{\rho}}_E / \rho_{\text{target}}) \rfloor}$) to reduce the effect of $\hat{\rho}$ outliers. The moving n showed a slight empirical advantage in our experiments.

This equalization yields subepoch records with similar expected error bounds across fragments, regardless of size or load, which facilitates accurate queries.

The Impact of ρ_{target} . One might be tempted to set ρ_{target} extremely low to maximize accuracy. However, ρ_{target} bounds error only *within a subepoch*, while queries span an entire epoch. In the presence of temporal blind spots, the epoch estimate extrapolates from the available subepochs, and its accuracy depends on how representative those measurements are. Thus the final error is $\epsilon = \epsilon_{subepoch} + \epsilon_{extrapolation} \approx \rho + \epsilon_{extrapolation}$. Because $n \propto 1/\rho_{target}$, lowering ρ_{target} reduces sketch noise but shortens subepochs, which increases extrapolation error.

Extrapolation accuracy is determined jointly by n and traffic burstiness: highly bursty flows are poorly represented by individual subepochs, while smoother flows extrapolate well. Choosing ρ_{target} therefore requires balancing per-subepoch error against burstiness. In Sec. IV-E, we propose a practical heuristic: an initial phase that measures this burstiness directly from counters, combined with an empirically calibrated mapping from burstiness to the ρ that minimizes query error.

The UnivMon Sketch. UnivMon is a versatile sketch consisting of multiple count sketches. According to the above, we set each fragment to contain all levels, all with the same width (as in the paper [58]) and the same subepoch hash.

C. Central Querying

Each fragment exports a subepoch record R at the end of each subepoch, which is sent for central collection.

A record is defined as $R = (F, E, S, n, \mathbf{c}, \mathbf{h})$ where:

- F is the fragment that the record is from,
- E is the epoch number,
- S is the subepoch number,
- n is the number of subepochs that F used in E ,
- $\mathbf{c}$ are the counters from the end of the subepoch,
- $\mathbf{h}$ are the hash functions used during sketching.

To refer to a field, e.g., F , inside a record R , we use $R.F$. Each R is added to the set of collected records $\mathbf{R}$, which forms the basis for network-wide queries.

A key-based query is defined as $Q = (\theta, \tau, \kappa)$ where:

- θ is the query type (e.g., flow frequency or data volume),
- $\tau = [T_{start}, T_{end}]$ is the query time window,
- κ is the queried key (e.g., a flow).

The set of supported query types and keys depends on the capabilities of the deployed sketches.

We assume queried windows align with sketching epochs, i.e., $Q.\tau$ spans contiguous epochs $\mathbf{E}_Q$, as is standard in most exported sketches [58]. Each epoch is queried to yield per-epoch outputs $O_E \in \mathbf{O}$ for each $E \in \mathbf{E}_Q$. These are merged to craft a final per-key output O_Q , for instance through $O_Q = \text{Sum}(\mathbf{O})$ or $O_Q = \text{Average}(\mathbf{O})$ depending on the query.

Each epoch E is queried as follows:

Step 1 - Retrieve relevant records: Let $\mathbf{R}_Q^E$ be the set of subepoch records that form the base for query Q in epoch E . The specific set depends on the query, but all records have E as their epoch number.

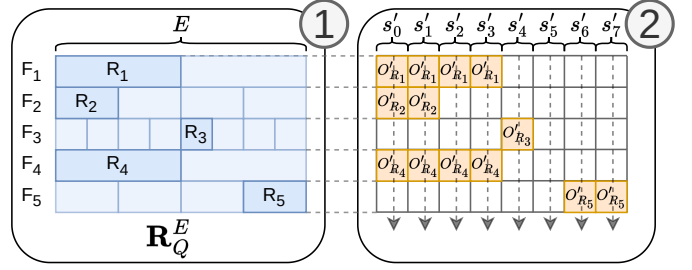


Fig. 8: Centralized querying. (1) Query-relevant records are retrieved that sampled the target key during the query window. (2) Normalized estimations are computed and merged.

Many queries, such as queries against a specific key κ , require knowledge of the network path $\mathbf{P}_\kappa$. We assume that the path for flows is known or computable. This is a standard assumption and holds under popular ECMP-based load balancing, where the underlying per-flow hashes can be deterministically recomputed by a central controller [31]. Moreover, auxiliary telemetry (e.g., INT) or routing state allows path inference even after the fact. In practice, operators routinely maintain sufficient routing, telemetry, and/or hashing information to recover the exact sequence of switches a flow has traversed. This path restricts $\mathbf{R}_Q^E$ so that $R.F \in \mathbf{P}_\kappa$.

Further, only records from subepochs that sampled κ are selected. This is done through the key-to-subepoch mapping hash function $s_E^F(\kappa)$, which is computed for each fragment $F \in \mathbf{P}_\kappa$, so that $R.S = s_E^F(\kappa)$. We now have a set $\mathbf{R}_Q^E$ containing all records within E that form the basis for query Q .

Step 2 - Query the Records: All records are normalized into equal-length subepochs to facilitate querying of varying-length subepoch records. Let n_m be the largest n for any record in $\mathbf{R}_Q^E$, which is the number of subepochs that records should normalize into.

Each record $R \in \mathbf{R}_Q^E$ is queried individually as a single-row sketch to retrieve its raw estimations O_R . For CMS, $O_R = c_i \in R.\mathbf{c}$, where $i = h_c(\kappa)$ and $h_c \in R.\mathbf{h}$ is the indexing hash used at sketching time. These estimates are divided into $N_R = n_m/R.n$ smaller estimates $O'_R = O_R/N_R$, one per normalized subepoch that overlaps with the record's subepoch.

To estimate a key's statistic during an epoch, we first estimate it for each normalized subepoch. For a given normalized subepoch S' , we consider all records that include S' and that sampled κ , and combine their estimates according to the sketch (e.g., minimum in Count-Min, median in Count Sketch).

As an example, suppose the queried key traverses five fragments and appears in records $R_1, \dots, R_5$ (Fig. 8, Step 1). Fragment F_3 has the highest subepoch count (8), so all records are normalized to $n_m = 8$ subepochs. In the first two normalized subepochs, the estimate combines $O'_{R_1}, O'_{R_2}, O'_{R_4}$; in the third, only O'_{R_1} and O'_{R_4} ; and so on.

Some normalized subepochs may lack estimates for κ (i.e., a temporal blind spot). In such cases, we use the mean of the other subepochs' estimates. For instance, in Fig. 8 Step 2, the key was not measured in the sixth subepoch, so we instead use the average of the other seven as that subepoch's estimation.

These blind spots cause extrapolation errors when that substitution is a poor proxy for the true value.

Finally, the sum of the normalized subepoch estimates yields the cumulative estimate for the entire epoch.

D. Enhanced Single-hop Sketching

As discussed in Sec. III and shown in Fig. 6, the accuracy of flow estimations depends on the number of fragments traversed. Single-hop flows perform worse because they do not benefit from redundant estimates, which is especially damaging when that fragment has relatively few counters. We therefore offer a mitigation strategy that improves single-hop accuracy at the cost of an accuracy degradation for other flows. We assume that fragments can identify single-hop flows at sketching time¹.

We enforce that these flows are monitored in two subepochs within each epoch. We modify the key-to-subepoch mapping to compute two subepochs $S_{E,0} = s_E(\kappa)$ and $S_{E,1} = s_E(\kappa) + n_E/2 \bmod n_E$. Thus, for $n_E \geq 2$, single-hop flows are monitored during two subepochs, one in each half of the epoch. If $n_E = 1$, this mitigation is not applicable, but it also suggests that the single-hop fragment is large enough to deliver suitably accurate estimates on its own.

Although this mitigation improves single-hop accuracy, it roughly doubles the counter increments for those flows, which amplifies noise and increases the fragment's ρ . We therefore expect increased errors in queries against other flows. We evaluate this effect in Sec. VI-D.

E. Heuristic ρ_{target} Selection

A flow's epoch error decomposes as:

$$\epsilon = \epsilon_{subepoch} + \epsilon_{extrapolation} \approx \rho + \epsilon_{extrapolation}.$$

Smaller ρ (more subepochs) lowers per-subepoch noise but shortens the observed window, which increases extrapolation error; larger ρ does the opposite. We therefore pick ρ_{target} by balancing *counts-per-subepoch* noise against *extrapolation* errors.

Warm-up burstiness estimation: At deployment, we run a short warm-up epoch with a large subepoch count (e.g., $n_{max} = 32$) and disable temporal sampling (i.e., every flow is tracked in every subepoch). Let $c_{i,s}$ be counter i at the end of subepoch s , and let $\Delta c_{i,s} = c_{i,s} - c_{i,s-1}$ be the increment in subepoch s . For each counter,

$$\mu_i = \frac{1}{n} \sum_{s=1}^n \Delta c_{i,s}, \quad \sigma_i^2 = \frac{1}{n-1} \sum_{s=1}^n (\Delta c_{i,s} - \mu_i)^2.$$

We merge per-counter burstiness into a median network-wide burstiness,

$$\sigma_{net} = \text{median}_i \{\sigma_i\}.$$

This subepoch-scale standard deviation (in counts) reflects the extrapolation variability when a single subepoch stands in for the epoch.

¹Single-hop flows can be identified, for example, when both a packet's ingress and egress ports leave the network, assuming all switches contain a sketching fragment.

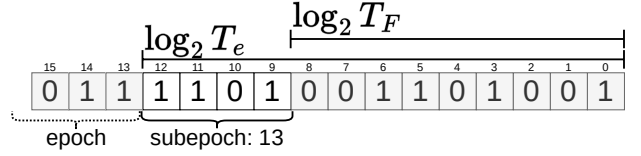


Fig. 9: Inferring the subepoch from a timestamp.

Empirical mapping from burstiness to ρ_{target} : We propose *learning* a mapping from burstiness to the best ρ using synthetic traffic. We suggest emulating synthetic workloads that span a range of burstiness levels (tuned to match σ_{net}) and, for each workload, sweep a grid of ρ values while measuring query error (e.g., RMSE) for the synthetic traffic. Let $\mathcal{R}$ be the tested ρ grid and let $\text{Err}(\rho; \sigma)$ denote the measured error under burstiness σ . We define a calibration function by

$$\rho^*(\sigma) = \arg \min_{\rho \in \mathcal{R}} \text{Err}(\rho; \sigma),$$

with ρ^* implemented as a lookup table with monotone interpolation over σ . At deployment time, we set

$$\rho_{target} \leftarrow \rho^*(\sigma_{net}).$$

After seeding ρ_{target} from ρ^* , fragments proceed with the equalization loop to adapt n so that their estimated per-subepoch PEBs $\hat{\rho}$ converge around ρ_{target} . In our experience, the system is forgiving to a suboptimal ρ_{target} , and fragments quickly converge on an effective subepoching.

V. IMPLEMENTATION

We provide two open-source implementations of DiSketch: a modular software simulator and a hardware prototype. The simulator supports flexible experimentation with disaggregated sketches across diverse topologies and configurations. The hardware prototype targets P4-programmable switches on commodity hardware, demonstrating feasibility at line-rate processing at multiple terabits per second with minimal resource overheads. Spatiotemporal disaggregation is optimized for the constraints of the Protocol-Independent Switch Architecture (PISA) [11], [12], enabling all packet processing to remain in the high-speed data plane.

A. Hardware Prototype

To demonstrate the hardware efficiency of our approach, we implement DiSketch on a P4-programmable switch. This implementation is deployed and verified on a Tofino 2 switch, capable of 6.4 Tbps network traffic. Our prototype is lightweight, comprising 850 lines of code split between the P4 data plane and the Python control plane.

Subepoch membership is computed entirely in the data plane using a bitslice of the switch-internal timestamp. Assuming epoch lengths and subepoch counts are powers of two, the intra-epoch offset is encoded in the lower $\log_2(\text{epoch_length})$ bits of the timestamp (Fig. 9). The currently active subepoch corresponds to the highest $\log_2(n)$ bits of this intra-epoch window, where n is the number of subepochs in the fragment.

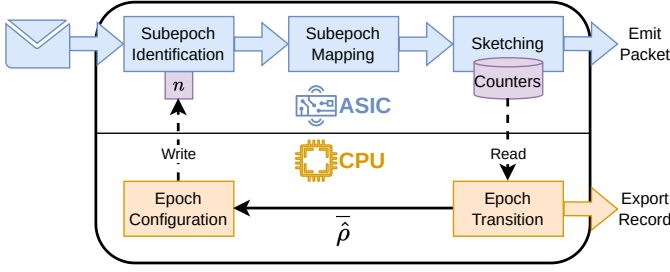


Fig. 10: An overview of the on-switch placement of DiSketch components. The high-speed ASIC does online sketching, while the slower on-switch CPU handles epoch transitions.

To determine whether a flow is active in the current subepoch, we hash the flow ID using a native CRC8 primitive and XOR the result with the intra-epoch timestamp. If the top $\log_2(n)$ bits of the result are all zero, the flow is selected for tracking. This bitwise match is implemented using a single-entry ternary match table, where the bitmask encodes the current value of n through its number of leading ones. This design supports low-latency reconfiguration and incurs minimal table space.

Epoch configuration is too complex to adhere to the strict PISA requirements and is deferred to the local control plane and invoked only at epoch boundaries, as shown in Fig. 10. To avoid counter resets, we track deltas: the controller queries raw counter values and subtracts the previously recorded state to infer per-subepoch activity. The only controller-ASIC communications required are counter exports and occasional bitmask updates when the number of subepochs changes.

Resource Efficiency: To evaluate hardware efficiency, we implemented both DiSketch and Distributed Sketch [35] on a Tofino ASIC [63]. Fig. 11 reports the resource footprints with 128 KiB counters. Distributed Sketch requires a per-flow lookup table to map flows into fragments, which causes its statically allocated ternary content-addressable memory (TCAM) requirements to grow linearly with the number of trackable flows. Using the authors’ official pipeline code², the system supports at most 1K flows before exceeding the single-stage TCAM limits; even without staging constraints, it cannot handle more than 6K flows without exhausting all TCAM resources. This cost is *static and essential*, and unrelated to the target accuracy.

In contrast, DiSketch imposes no such cap: it can monitor an arbitrary number of flows, with accuracy determined by the number of counters and intra-flow burstiness. Its overheads are modest and *static*, confined mainly to hash distribution units and VLIW instructions, which makes the design both efficient and scalable. Importantly, these costs are static, and scaling DiSketch to higher accuracy or more flows increases only the SRAM footprint, without inflating other hardware resources.

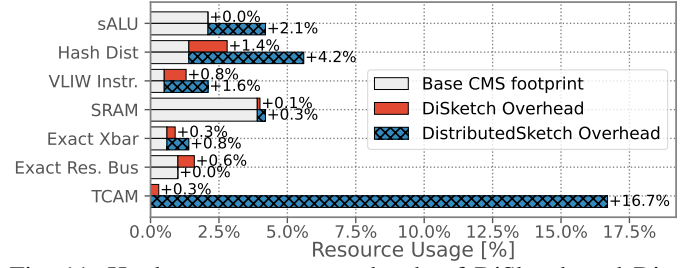


Fig. 11: Hardware resource overheads of DiSketch and Distributed Sketch [35], using CMS as the underlying sketch. Note that Distributed Sketch here supports *at most* 1K flows.

VI. EVALUATION

Topologies: We investigate deployments in two widely used datacenter topologies that dominate the industry: the Fat-Tree and SpineLeaf topologies. In our simulations, we configure these as a $k = 2$ Fat-Tree with four core switches (20 switches in total) and a spine-leaf fabric consisting of 12 switches³. Together, these topologies cover the most common deployment scenarios in modern datacenters and capture the path diversity and hop-count variation relevant to our evaluation [5], [44], [51], [71].

Memory Distributions: In these networks, we evaluate two switch memory size distributions: *Homogeneous* and *Heterogeneous*. The heterogeneous scenario allocates pseudo-random memory sizes for per-switch sketches. These memories are generated with a pre-defined average memory size and inter-switch heterogeneity, designed to correspond with a selected heterogeneity level⁴. The generated memories are randomly distributed to switches, not considering their topological position. As a default, we use an arbitrary heterogeneity level of Gini = 0.4, resulting in notable memory size variation between switches. An example memory distribution for five switches with Gini = 0.4 can be: [10%, 30%, 100%, 160%, 200%], i.e., fragment-sizes ranging from 10% to 200% of the base memory size. The homogeneous scenario allocates exactly the base memory size for all switches, resulting in a heterogeneity level of 0.

Sketches: We are evaluating spatiotemporally disaggregated instances of Count Sketch (CS), Count-Min Sketch (CMS), and UnivMon (UM)⁵. We do not compare against Distributed Sketch [35] due to its inability to track all flows (see Sec. V). To simplify the discussion, we name applied versions of our solution as *DiSketches* (Disaggregated Sketches), i.e., DiSketch-CS, DiSketch-CMS, and DiSketch-UM. For comparison, we also evaluate aggregated sketches on core switches in the network, and DISCO [16] deployed network-wide. Fragments utilize all available memory of their respective switches.

³The simulated networks are small in size due to computational restrictions. For context, a single data point in our experiments requires nearly a full hour of simulation time. We have simulated topologies up to approximately double these sizes, without any noticeable impact on the results. Simultaneously, we see no realistic scenario where the topology scale would degrade the accuracy of DiSketch.

⁴The Gini inequality index is used to generate random memory distributions.

⁵We deploy UnivMon with 16 levels, which is approximately the second logarithm of the expected number of flows.

²Find their code at <https://github.com/DpintHPCC416/Distribute/>

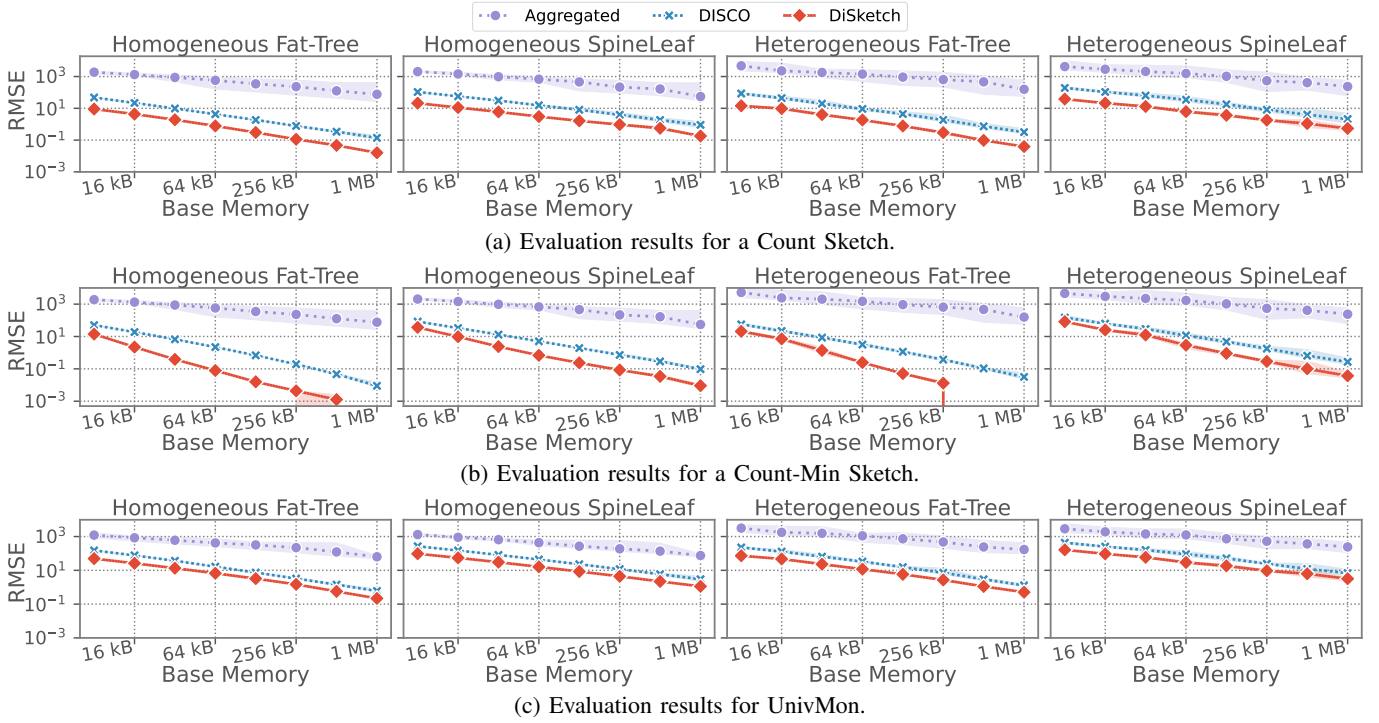


Fig. 12: Frequency estimation errors of sketches for 5-hop flows at various memory sizes. Scenarios are combinations of the Fat-Tree and SpineLeaf topologies, either with homogeneous or heterogeneous switch memory sizes.

Traces: In this evaluation, we use the real-world CAIDA-NYC Equinix packet trace [17], recorded at a backbone link in 2019. If nothing else is stated, then we replay ~ 5 seconds of traffic (~ 2 M packets, covering ~ 200 K flows). We only have access to traffic recorded at a single network link, and we map IP addresses in traffic uniformly at random to hosts in our network to simulate network-wide communication, while omitting flows where both the source and destination map to the same host.

DiSketch converges on an optimal epoch configuration over time, and we split the query window into arbitrary 32 epochs to allow DiSketch to converge. We do, however, include the suboptimal initial epochs in the query window.

A. Frequency Estimation Errors

In Fig. 12, we present the root mean squared errors (RMSEs) of flow frequency estimation, a common sketch query [18], [24], [58]. Here, we query all flows that were tracked during the experiment in terms of their number of transmitted packets and define the query error as the absolute difference from the ground truth number of packets. We use the aforementioned experimental setup and replay traffic across the entire network topology. In this figure, only flows with full-length network paths are queried, to ensure that all evaluated flows would have traversed an aggregated sketch residing on core switches. We evaluate per-path-length performance later in Sec. VI-D.

This evaluation shows a clear pattern: our solution consistently outperforms both aggregated and DISCO-disaggregated versions of all three sketches, in both homogeneous and heterogeneous topologies.

As an example, in a heterogeneous Fat-Tree deployment, DiSketch-CS delivers $RMSE < 1$ using on average 128KB of per-switch memory, which is approximately 25% of the 512KB that DISCO requires. All aggregated sketches failed to deliver this accuracy under the evaluated memory constraints. Alternatively, using the same 512KB of average per-switch memory, the aggregated CS, DISCO-CS, and DiSketch-CS deliver $RMSE$ of 460, 0.8, and 0.09, respectively. These patterns hold when the sketching epoch, or the underlying traffic volumes, increase. We see similar patterns for all evaluated sketches and network topologies. This does, however, only investigate flows with full-length network paths, at fixed heterogeneity levels. We will demonstrate the effect of these parameters in sections VI-C and VI-D.

Takeaway: DiSketch consistently and significantly enhances the memory-vs-accuracy tradeoff for frequency estimation, either increasing the accuracy by nearly an order of magnitude or reducing the required memory by approximately 75% depending on the sketching environment.

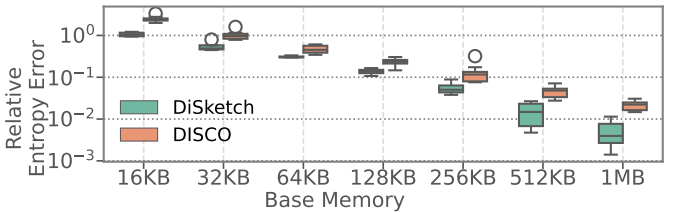


Fig. 13: The relative entropy error of UnivMon in a Fat-Tree network with a random memory distribution.

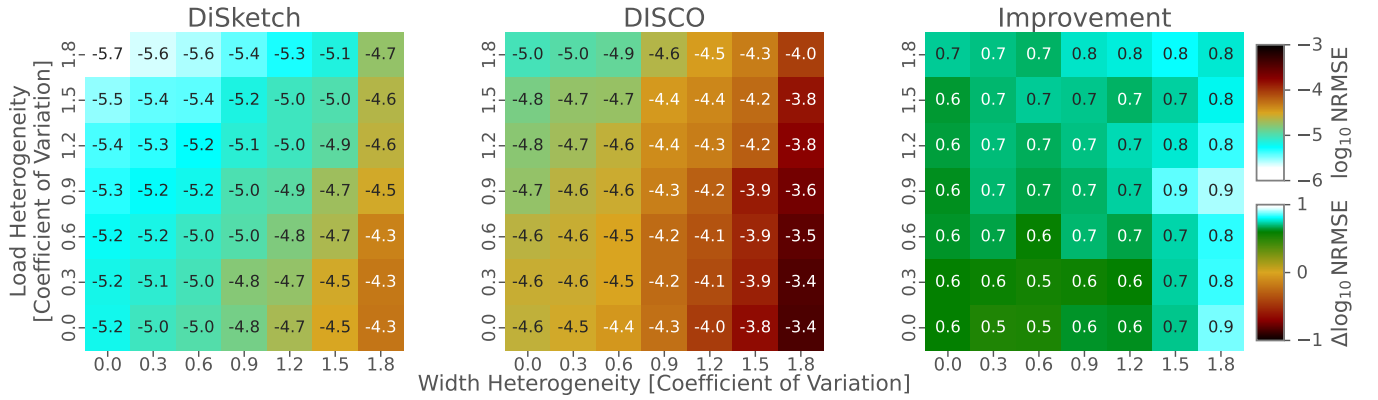


Fig. 14: Heterogeneity’s impact on frequency estimation, showing the $\log_{10}(\text{NRMSE})$ at various heterogeneity combinations. The right-most heatmap shows the $\log_{10}(\text{NRMSE})$ improvement over DISCO gained through spatiotemporal disaggregation.

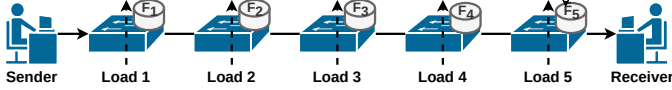


Fig. 15: Experimental Setup in Heterogeneity Tests.

B. Entropy Estimation Errors

In Fig. 13, we present the experimental results of estimating the network-wide entropy of IP addresses. This is done through UnivMon, according to the algorithm outlined in their paper [58]. Given that disaggregated sketches far outperform aggregated sketches, we here compare DISCO only with DiSketch. We therefore also remove the full-path-length limitation and base these evaluations on all network traffic. We present only the heterogeneous Fat-Tree scenario here, but the results are similar in all evaluated scenarios.

Similar to the previous section, we note a consistent and significant error reduction through our solution, and DiSketch-UM delivers a similar entropy estimation accuracy as DISCO-UM using half as much memory. Alternatively, DiSketch-UM lowers the errors by approximately 50% across tests.

Takeaway: Our solution halves the amount of memory required for entropy estimation, or halves the errors while keeping the memory unchanged.

C. Heterogeneity Effects

Here, we evaluate DiSketch in heterogeneous environments where traffic load and memory size vary across switches. For simplicity, we again focus on the frequency estimation accuracy of DISCO-CS and DiSketch-CS.

Experimental Setup: As opposed to the prior experiment, this experiment simulates a single 5-hop network path as shown in Fig. 15. The network load heterogeneity depends on various factors, including the network topology, load balancing, node positions, and the traffic patterns of connected hosts. By simulating a single path, we gain precise control of the per-switch traffic volumes and can set the heterogeneity levels freely. Traffic on other semi-overlapping paths is emulated as per-switch background traffic.

We vary the memory and traffic load heterogeneity of the hops, according to the coefficient of variation (CoV, i.e., the relative standard deviation). Heterogeneity levels range from perfectly homogeneous (CoV = 0) to significantly heterogeneous (CoV = 1.8). The total amount of background traffic (259K packets) and the number of on-path counters (5120) is fixed across tests⁶. Two pseudo-random lists of 5 integers are generated for each test, one with the per-hop load and the other with the per-fragment memory size, so that both lists adhere to the experimental parameters. For instance, an example background traffic distribution with CoV ≈ 1.5 is [204189(78.7%), 18364(7.1%), 29(0.01%), 2265(0.9%), 34675(13.4%)]. The generated loads and widths are independent and are randomly distributed to the nodes.

There are six different packet streams: five background traffic streams, each passing through one switch, and one evaluation stream that traverses the entire path. The aforementioned CAIDA packet trace serves as the basis for all network traffic, and we randomly map IP addresses into the traffic streams. Packets in each stream are replayed chronologically in the order they appear in the packet trace. Approximately 300 flows comprising approximately 1% of the total traffic are replayed across the full path, from sender to receiver, and are used for evaluation. The remaining 99% of traffic only *crosses* the individual nodes (see Load in Fig. 15). The experimental results are only based on flows traversing all five of these fragments.

The experimental results are provided in terms of the Normalized Root Mean Squared Error (NRMSE) [7] between the estimated frequencies and the known ground truth, and are normalized by the total number of packets to provide a dimensionless measure of error. A lower NRMSE indicates better performance.

Results. The average results following multiple simulations are presented in Fig. 14 for all heterogeneity combinations. To further help illustrate the effect of DiSketch, we present the difference in NRMSE between DiSketch and DISCO in the rightmost heatmap.

⁶The small scale in these simulations allowed us to evaluate numerous heterogeneity settings within a reasonable time, and yields the same general pattern as full-scale heterogeneity simulations.

Fig. 14 reveals several patterns, most notably a clear link between heterogeneity and sketching accuracy. The impact of width heterogeneity is expected: greater variation in per-switch memory reduces overall accuracy. This follows from sketch theory, where a single row may be inaccurate but combining multiple rows improves accuracy superlinearly. Independent rows act as repetitions that suppress collision noise; when capacity is concentrated in a few switches, the number of effective rows along typical paths falls, degrading accuracy under a fixed total memory budget. However, the apparent improvement from load heterogeneity is likely an experimental artifact. Because we evaluate only full-path flows while keeping total background traffic constant, pushing more load onto fewer switches reduces noise on the remaining on-path fragments, boosting their accuracy. This effect would likely disappear if the accuracy of background flows were included in the evaluation.

The right-most heatmap highlights the accuracy improvement when replacing DISCO with DiSketch. Accuracy consistently improves across all heterogeneity combinations, with larger gains as heterogeneity increases. This is expected: DISCO is heterogeneity-unaware, whereas DiSketch fragments adaptively reconfigure to equalize the network-wide PEB. Note that the improvement is shown as the absolute change in $\log_{10}$ NRMSE, where a value of 1 signals an order of magnitude NRMSE reduction.

Takeaway: DiSketch outperforms DISCO in all heterogeneity combinations tested. This is especially evident as the heterogeneity levels increase, demonstrating the effectiveness of network-wide PEB equalization.

D. Path Length Effects

Here, we evaluate how path lengths affect the accuracy of DiSketch-CS and DISCO-CS. Using the heterogeneous Fat-Tree scenario from the previous section, we group query results by flow path length. For clarity, Fig. 16 reports root mean squared errors (RMSEs) only for the smallest (8KB) and largest (1MB) base memories.

As expected, longer paths (i.e., more fragments) improve accuracy across all disaggregated sketches. The effect is stronger with more memory: at 1MB, single-hop flows incur about $50\times$ higher error than 3-hop flows, compared to $2.8\times$ at 8KB. DiSketch also consistently outperforms DISCO across all path lengths. Applying the single-hop mitigation from Sec. IV-D reduces single-hop error by 24% (8KB) and 13% (1MB), while additional counter noise slightly increases error for multi-hop flows. However, both of these effects are mild and remain within one standard deviation of an unmitigated deployment.

Takeaway: The query accuracy is greatly impacted by the path length of the underlying flow, and the single-hop mitigation results in a slight accuracy enhancement for single-hop flows and a slight accuracy decrease for multi-hop flows. Accordingly, whether mitigation is worthwhile may depend on the setting.

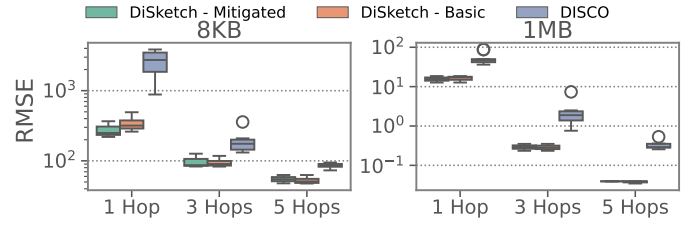


Fig. 16: The path lengths’ impact on frequency estimation accuracy for Count Sketch. DiSketch is evaluated both with and without the single-hop mitigation strategy.

VII. DISCUSSION

Additional Structures: We demonstrated spatiotemporal disaggregation with Count Sketch, Count-Min Sketch, and UnivMon. We expect the same principles to extend to other sketches (e.g., Bloom filters and HyperLogLog) and even to non-sketch data structures. Identifying the precise technical requirements for spatiotemporal disaggregability and the modifications needed for each structure is an open direction for future work. However, non-cumulative estimators may require adjustments to the temporal merging of subepochs, while spatial merging of fragments typically follows the row-merging logic of the non-aggregated sketch (e.g., Count Sketch using the median).

Temporal Dynamics and Pathing: Load balancing and network dynamics introduce time-varying conditions that complicate disaggregation. Schemes such as flowlet switching [72] may change paths faster than epoch timescales, and failures can reroute traffic within an epoch. If increments from a flow are split across fragments mid-epoch, estimates may become biased unless the analysis engine is path-aware. One option is to merge all fragments traversed in the epoch, weighting each by the time a flow spent there. An operational alternative is to restrict rerouting to epoch boundaries (except on failures), ensuring that each epoch uses a consistent fragment set. Transient failures with missing records are already handled by our blind-spot tolerant query procedure (Sec. IV-C).

Resource availability can also vary as co-located functions consume or release memory. Rather than treating this solely as noise, DiSketch can capitalize on such fluctuations through PEB equalization, where each fragment adjusts its subepoch count n based on recent accuracy estimates. This closed-loop adaptation enables fragments to exploit newly available memory to reduce errors and to scale down gracefully when memory shrinks.

Finding an Optimal ρ_{target} : The effectiveness of spatiotemporal disaggregation depends on balancing subepoch noise against extrapolation error, a tradeoff governed by ρ_{target} . Our design (§IV-E) provides a heuristic for seeding this value from observed burstiness, after which fragments converge through equalization. Although the system is robust to moderate misconfiguration, a deeper theoretical characterization of the optimal ρ_{target} and approaches to eliminate this hyperparameter altogether remain important directions for future work.

Collection Cost: Subepoching increases the export rate, since each fragment emits one record per subepoch rather than one per epoch. In our deployments, the network-wide average subepoch count n converges to single digits (often $n \approx 5$), yielding about a $5\times$ increase in exported records. However, the per-record payload remains small: a fixed header plus w_F counters (up to 128 Ki in our hardware). Because DiSketch exports only counter deltas over shorter observation windows, counters can be narrower without risk of overflow. Using 16 bits instead of 32 bits halves the payload size and keeps collection read-only on the data plane.

To place the rate in context, sketching epochs are long relative to data plane timescales (often 1 s). A per-switch export interval of 100 ms is therefore reasonable; in a 1,000-switch network, this corresponds to roughly 10,000 records per second system-wide. Modern telemetry collectors comfortably ingest orders of magnitude more (on the order of 10^8 reports/s) [48]. While operators should balance the accuracy gains from higher n against the added collection load, the overhead at datacenter scale remains modest.

VIII. RELATED WORK

Traditionally, sketches have been deployed in an aggregated manner [18], [24], [58]. Recent interest has shifted towards network-wide deployments to enhance measurement flexibility [16], [26], [35], [53], [77], [80].

For example, Zhao et al., 2021, [80] designed a sketch-based network-wide telemetry system named LightGuardian. In their approach, each switch hosts two *SuMax* sketches: one actively populated and one being collected. This supports new sketch measurements, including latency jitter and packet loss detection, using a probabilistic in-band collection method to reduce centralized collection costs. Nonetheless, this system does not address heterogeneous environments, incurs substantial resource costs, and employs aggregated (i.e., non-disaggregated) sketches on each switch.

To our knowledge, the first paper describing the potential of disaggregated sketches was DISCO [16], published in 2020. They argued for sketch disaggregation to simplify sketch deployments in resource-scarce environments. They presented a basic technique for per-row disaggregation of sketches for estimating flow size, showing an increased accuracy in heavy hitter detection. Although promising, they did not investigate heterogeneous environments, hardware feasibility, or more complex sketches for tasks unrelated to flow size estimation.

Cornacchia et al. (2021), [26] highlighted the detrimental effects of traffic patterns on per-row disaggregated sketches, notably increased hash collisions and accuracy degradation due to load imbalances. They proposed that sketch fragments sample a subset of traffic to process, but their algorithm assumes full in-band knowledge of per-flow paths and fragment dimensions, thus introducing considerable overhead and limiting deployment flexibility. We believe that their fundamental idea is valid, that sampling techniques would lead to efficient distributed sketching in heterogeneous environments, but that their imposed assumptions are unreasonable in many deployments. Hence, we choose not to include this solution

in the evaluation. DiSketch fragments operate autonomously, without any per-flow knowledge assumptions. We discussed the issues with these assumptions in Sec. III.

Gu et al. (2023), [35] proposed per-column disaggregation as an alternative to per-row disaggregation and discussed handling load imbalances. As outlined in Sec. III, this approach faces the challenge of evenly distributing counters across diverse paths to ensure balanced counter incrementation relative to fragment sizes. To address this, the authors propose lookup tables for counter allocation, with one entry for every active traffic flow. However, since the number of sketch cells typically grows sub-linearly to the number of keys, including such a lookup table contradicts this fundamental design goal. Further, per-column disaggregation is inefficient in high-performance switching architectures such as PISA, since per-row computational logic remains dedicated even for packets where it is not utilized, leading to a high resource footprint.

Li et al. (2024), [53], address the traffic imbalance issue by proposing a deployment and increment strategy that ensures load balancing across sketch rows. Their method selectively deploys rows across the network, supporting a variable number of rows per fragment. The ingress switch determines the number of rows each hop should process per packet, inserting this information as a new header for ingressing packets and allocating traffic based on hop capacities. However, this places high burdens on ingress switches, requiring extensive knowledge of network paths and fragment dimensions, and reduces the MTU of the network through extended packet headers, risking frame fragmentation and reduced goodput. DiSketches has none of these issues.

IX. CONCLUSION

We introduced spatiotemporal sketch disaggregation, a novel sketching scheme for disaggregated streaming analysis. This approach combines spatial and temporal indexing through subepoching, enabling flexible and heterogeneous deployments of multiple sketches, as demonstrated with Count Sketch, Count-Min Sketch, and UnivMon. Our findings show that spatiotemporal disaggregation significantly enhances sketch accuracy, reducing estimation errors by nearly an order of magnitude compared to DISCO, and by several orders of magnitude over conventional aggregated sketching methods. These accuracy improvements are especially pronounced in heterogeneous environments, where network-wide equalization of probabilistic error bounds allows fragments of varying loads and sizes to be efficiently queried together.

ACKNOWLEDGEMENTS

This work has been partially supported by KTH Digital Futures.

REFERENCES

- [1] Marcos K. Aguilera, Wojciech Golab, and Mehul A. Shah. A practical scalable distributed b-tree. *Proc. VLDB Endow.*, 1(1), 2008.
- [2] Mohammad Al-Fares, Alexander Loukissas, and Amin Vahdat. A scalable, commodity data center network architecture. *ACM SIGCOMM computer communication review*, 38(4):63–74, 2008.

- [3] Mohammad Alizadeh, Tom Edsall, Sarang Dharmapurikar, Ramanan Vaidyanathan, Kevin Chu, Andy Fingerhut, Vinh The Lam, Francis Matus, Rong Pan, Navindra Yadav, et al. Conga: Distributed congestion-aware load balancing for datacenters. In *Proc. of ACM SIGCOMM*, 2014.
- [4] Noga Alon, Yossi Matias, and Mario Szegedy. The space complexity of approximating the frequency moments. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pages 20–29, 1996.
- [5] Alexey Andreyev. Introducing data center fabric, the next-generation facebook data center network, 2014.
- [6] Ran Ben Basat, Gil Einziger, Roy Friedman, Marcelo Caggiani Luizelli, and Erez Waisbard. Volumetric hierarchical heavy hitters. In *Proc. of IEEE MASCOTS*, 2018.
- [7] Ran Ben Basat, Gil Einziger, Michael Mitzenmacher, and Shay Vargafik. Salsa: self-adjusting lean streaming analytics. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*, pages 864–875. IEEE, 2021.
- [8] Ran Ben Basat, Sivaramakrishnan Ramanathan, Yuliang Li, Gianni Antichi, Minlan Yu, and Michael Mitzenmacher. PINT: probabilistic in-band network telemetry. In *SIGCOMM '20*, pages 662–680. ACM, 2020.
- [9] Theophilus Benson, Aditya Akella, and David A Maltz. Network traffic characteristics of data centers in the wild. In *Proc. of SIGCOMM IMC*, 2010.
- [10] Burton H Bloom. Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13(7):422–426, 1970.
- [11] Pat Bosshart, Dan Daly, Glen Gibb, Martin Izzard, Nick McKeown, Jennifer Rexford, Cole Schlesinger, Dan Talayco, Amin Vahdat, George Varghese, et al. P4: Programming protocol-independent packet processors. *ACM SIGCOMM CCR*, 2014.
- [12] Pat Bosshart, Glen Gibb, Hun-Seok Kim, George Varghese, Nick McKeown, Martin Izzard, Fernando Mujica, and Mark Horowitz. Forwarding metamorphosis: Fast programmable match-action processing in hardware for sdn. *ACM SIGCOMM Computer Communication Review*, 43(4):99–110, 2013.
- [13] Broadcom. Broadcom NetOps. <https://academy.broadcom.com/blog/network-operations/dx-netops/5-steps-to-get-telemetry-data-in-dx-netops>.
- [14] Broadcom. Broadcom Tomahawk 5. <https://www.broadcom.com/products/ethernet-connectivity/switching/strataxgs/bcm78900-series>.
- [15] Broadcom. Broadcom Trident 5. <https://www.broadcom.com/products/ethernet-connectivity/switching/strataxgs/bcm78800>.
- [16] Valerio Bruschi, Ran Ben Basat, Zaoxing Liu, Gianni Antichi, Giuseppe Bianchi, and Michael Mitzenmacher. Discovering the heavy hitters with disaggregated sketches. In *Proceedings of the 16th International Conference on emerging Networking EXperiments and Technologies*, pages 536–537, 2020.
- [17] CAIDA. Passive monitor: equinix-nyc, 2019.
- [18] Moses Charikar, Kevin Chen, and Martin Farach-Colton. Finding frequent items in data streams. In *Proc. of ICALP*, 2002.
- [19] Peiqing Chen, Dong Chen, Lingxiao Zheng, Jizhou Li, and Tong Yang. Out of many we are one: Measuring item batch with clock-sketch. In *Proceedings of the 2021 International Conference on Management of Data*, pages 261–273, 2021.
- [20] Peiqing Chen, Yuhuan Wu, Tong Yang, Junchen Jiang, and Zaoxing Liu. Precise error estimation for sketch-based flow measurement. In *Proceedings of the 21st ACM Internet Measurement Conference*, pages 113–121, 2021.
- [21] Peter Clifford and Ioana Cosma. A simple sketching algorithm for entropy estimation over streaming data. In *Artificial Intelligence and Statistics*, pages 196–206. PMLR, 2013.
- [22] Edith Cohen, Rasmus Pagh, and David Woodruff. Wor and p’s: Sketches for l_p -sampling without replacement. *Advances in Neural Information Processing Systems*, 33:21092–21104, 2020.
- [23] Michael B Cohen and Richard Peng. Lp row sampling by lewis weights. In *Proceedings of the forty-seventh annual ACM symposium on Theory of computing*, pages 183–192, 2015.
- [24] Graham Cormode and S. Muthukrishnan. An Improved Data Stream Summary: The Count-Min Sketch and Its Applications. *J. Algorithms*, 2005.
- [25] Graham Cormode and Shan Muthukrishnan. An improved data stream summary: the count-min sketch and its applications. *Journal of Algorithms*, 55(1):58–75, 2005.
- [26] Alessandro Cornacchia, German Sviridov, Paolo Giaccone, and Andrea Bianco. A traffic-aware perspective on network disaggregated sketches. In *2021 19th Mediterranean Communication and Computer Networking Conference (MedComNet)*, pages 1–4. IEEE, 2021.
- [27] Christina Delimitrou, Sriram Sankar, Aman Kansal, and Christos Kozyrakis. Echo: Recreating network traffic maps for datacenters with tens of thousands of servers. In *2012 IEEE International Symposium on Workload Characterization (IISWC)*, pages 14–24. IEEE, 2012.
- [28] Fenghao Dong, Yang He, Yutong Liang, Zirui Liu, Yuhuan Wu, Peiqing Chen, and Tong Yang. Simisketch: Efficiently estimating similarity of streaming multisets. *arXiv preprint arXiv:2405.19711*, 2024.
- [29] Cristian Estan and George Varghese. New directions in traffic measurement and accounting. In *Proceedings of the 2002 conference on Applications, technologies, architectures, and protocols for computer communications*, pages 323–336, 2002.
- [30] Bin Fan, Dave G Andersen, Michael Kaminsky, and Michael D Mitzenmacher. Cuckoo filter: Practically better than bloom. In *Proceedings of the 10th ACM International Conference on emerging Networking Experiments and Technologies*, pages 75–88, 2014.
- [31] Adithya Gangidi, Rui Miao, Shengbao Zheng, Sai Jayesh Bondu, Guilherme Goes, Hany Morsy, Rohit Puri, Mohammad Riftadi, Ashmitha Jeevaraj Shetty, Jingyi Yang, et al. Rdma over ethernet for distributed training at meta scale. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 57–70, 2024.
- [32] Sumit Ganguly. Counting distinct items over update streams. *Theoretical Computer Science*, 378(3):211–222, 2007.
- [33] Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung. The google file system. In *Proc. of ACM SOSP*, 2003.
- [34] Soudeh Ghorbani, Zibin Yang, P Brighton Godfrey, Yashar Ganjali, and Amin Firoozshahian. Drill: Micro load balancing for low-latency data center networks. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, pages 225–238, 2017.
- [35] Liyuan Gu, Ye Tian, Wei Chen, Zhongxiang Wei, Cenman Wang, and Xinming Zhang. Per-flow network measurement with distributed sketch. *IEEE/ACM Transactions on Networking*, 2023.
- [36] Arpit Gupta, Rob Harrison, Marco Canini, Nick Feamster, Jennifer Rexford, and Walter Willinger. Sonata: Query-driven streaming network telemetry. In *Proc. of ACM SIGCOMM*, 2018.
- [37] Rob Harrison, Qizhe Cai, Arpit Gupta, and Jennifer Rexford. Network-wide heavy hitter detection with commodity switches. In *Proceedings of the Symposium on SDN Research*, pages 1–7, 2018.
- [38] Nicholas JA Harvey, Jelani Nelson, and Krzysztof Onak. Sketching and streaming entropy via approximation theory. In *2008 49th Annual IEEE Symposium on Foundations of Computer Science*, pages 489–498. IEEE, 2008.
- [39] Yongchao He, Wenfei Wu, Xuemin Wen, Haifeng Li, and Yongqiang Yang. Scalable on-switch rate limiters for the cloud. In *IEEE INFOCOM 2021-IEEE Conference on Computer Communications*, pages 1–10. IEEE, 2021.
- [40] Qun Huang, Xin Jin, Patrick P. C. Lee, Runhui Li, Lu Tang, Yi-Chao Chen, and Gong Zhang. Sketchvisor: Robust network measurement for software packet processing. In *Proc. of ACM SIGCOMM*, 2017.
- [41] Qun Huang, Patrick PC Lee, and Yungang Bao. Sketchlearn: Relieving user burdens in approximate measurement with automated statistical inference. In *Proc. of ACM SIGCOMM*, 2018.
- [42] Qun Huang, Siyuan Sheng, Xiang Chen, Yungang Bao, Rui Zhang, Yanwei Xu, and Gong Zhang. Toward nearly-zero-error sketching via compressive sensing. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, pages 1027–1044, 2021.
- [43] Piotr Indyk and Milan Ruzic. Near-optimal sparse recovery in the l_1 norm. In *2008 49th Annual IEEE Symposium on Foundations of Computer Science*, pages 199–207. IEEE, 2008.
- [44] Nikhil Jain, Abhinav Bhatele, Louis H Howell, David Böhme, Ian Karlin, Edgar A León, Misbah Mubarak, Noah Wolfe, Todd Gamblin, and Matthew L Leininger. Predicting the performance impact of different fat-tree configurations. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–13, 2017.
- [45] Naga Katta, Aditi Ghag, Mukesh Hira, Isaac Keslassy, Aran Bergman, Changhoon Kim, and Jennifer Rexford. Clove: Congestion-aware load balancing at the virtual edge. In *Proceedings of the 13th International Conference on emerging Networking EXperiments and Technologies*, pages 323–335, 2017.

- [46] Naga Katta, Mukesh Hira, Changhoon Kim, Anirudh Sivaraman, and Jennifer Rexford. Hula: Scalable load balancing using programmable data planes. In *Proceedings of the Symposium on SDN Research*, pages 1–12, 2016.
- [47] Xsight Labs. X-Switch: Programmable Ethernet Switches. <https://xsightlabs.com/wp-content/uploads/2024/09/X2-PB-Public.pdf>.
- [48] Jonatan Langlet, Ran Ben Basat, Gabriele Oliaro, Michael Mitzenmacher, Minlan Yu, and Gianni Antichi. Direct telemetry access. In *Proceedings of the ACM SIGCOMM 2023 Conference*, pages 832–849, 2023.
- [49] ChonLam Lao, Yanfang Le, Kshiteej Mahajan, Yixi Chen, Wenfei Wu, Aditya Akella, and Michael Swift. ATP: In-network aggregation for multi-tenant learning. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, pages 741–761. USENIX Association, April 2021.
- [50] Kasper Green Larsen, Rasmus Pagh, and Jakub Tětek. Countsketches, feature hashing and the median of three. In *International Conference on Machine Learning*, pages 6011–6020. PMLR, 2021.
- [51] Brian Lebiednik, Aman Mangal, and Niharika Tiwari. A survey and evaluation of data center network topologies. *arXiv preprint arXiv:1605.01701*, 2016.
- [52] Alberto Lerner, Rana Hussein, Philippe Cudré-Mauroux, and U eXascale Infolab. The case for network accelerated query processing. In *CIDR*, 2019.
- [53] Fuliang Li, Kejun Guo, Jiaxing Shen, and Xingwei Wang. Effective network-wide traffic measurement: A lightweight distributed sketch deployment. In *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*, 2024.
- [54] Minghao Li, Ran Ben Basat, Shay Vargaftik, ChonLam Lao, Kevin Xu, Xinran Tang, Michael Mitzenmacher, and Minlan Yu. THC: Accelerating Distributed Deep Learning Using Tensor Homomorphic Compression. In *USENIX Symposium on Networked Systems Design and Implementation*, 2024.
- [55] Yuliang Li, Rui Miao, Mohammad Alizadeh, and Minlan Yu. {DETER}: Deterministic {TCP} replay for performance diagnosis. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, pages 437–452, 2019.
- [56] Yuliang Li, Rui Miao, Hongqiang Harry Liu, Yan Zhuang, Fei Feng, Lingbo Tang, Zheng Cao, Ming Zhang, Frank Kelly, Mohammad Alizadeh, et al. Hpc: High precision congestion control. In *Proceedings of the ACM Special Interest Group on Data Communication*, pages 44–58, 2019.
- [57] Zaoxing Liu, Zhihao Bai, Zhenming Liu, Xiaozhou Li, Changhoon Kim, Vladimir Braverman, Xin Jin, and Ion Stoica. Distcache: Provable load balancing for large-scale storage systems with distributed caching. In *Proc. of USENIX FAST*, 2019.
- [58] Zaoxing Liu, Antonis Manousis, Gregory Vorsanger, Vyas Sekar, and Vladimir Braverman. One sketch to rule them all: Rethinking network flow monitoring with univmon. In *Proc. of ACM SIGCOMM*, 2016.
- [59] Zaoxing Liu, Hun Namkung, Georgios Nikolaidis, Jeongkeun Lee, Changhoon Kim, Xin Jin, Vladimir Braverman, Minlan Yu, and Vyas Sekar. Jaqen: A high-performance switch-native approach for detecting and mitigating volumetric ddos attacks with programmable switches. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 3829–3846, 2021.
- [60] Rui Miao, Hongyi Zeng, Changhoon Kim, Jeongkeun Lee, and Minlan Yu. Silkroad: Making stateful layer-4 load balancing fast and cheap using switching asics. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, pages 15–28, 2017.
- [61] Edgar Costa Molero, Stefano Vissicchio, and Laurent Vanbever. Fast in-network gray failure detection for isps. In *Proceedings of the ACM SIGCOMM 2022 Conference*, pages 677–692, 2022.
- [62] Andrea Monterubbiano, Jonatan Langlet, Stefan Walzer, Gianni Antichi, Pedro Reviriego, and Salvatore Pontarelli. Lightweight acquisition and ranging of flows in the data plane. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 7(3):1–24, 2023.
- [63] Barefoot Networks. Barefoot Tofino. <https://barefootnetworks.com/products/brief-tofino/>.
- [64] NVIDIA. NVIDIA DeepStream. https://docs.nvidia.com/metropolis/deepstream/dev-guide/text/DS_Overview.html.
- [65] Tian Pan, Nianbing Yu, Chenhao Jia, Jianwen Pi, Liang Xu, Yisong Qiao, Zhiguo Li, Kun Liu, Jie Lu, Jianyuan Lu, et al. Sailfish: Accelerating cloud-scale multi-tenant multi-service gateways with programmable switches. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*, pages 194–206, 2021.
- [66] Arjun Roy, Hongyi Zeng, Jasmeet Bagga, George Porter, and Alex C Snoeren. Inside the social network’s (datacenter) network. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*, pages 123–137, 2015.
- [67] Amedeo Sapia, Marco Canini, Chen-Yu Ho, Jacob Nelson, Panos Kalnis, Changhoon Kim, Arvind Krishnamurthy, Masoud Moshref, Dan RK Ports, and Peter Richtárik. Scaling distributed machine learning with in-network aggregation. *arXiv preprint arXiv:1903.06701*, 2019.
- [68] Brandon Schlinker, Italo Cunha, Yi-Ching Chiu, Srikanth Sundaresan, and Ethan Katz-Bassett. Internet performance from facebook’s edge. In *Proceedings of the Internet Measurement Conference*, pages 179–194, 2019.
- [69] Vibhaalakshmi Sivaraman, Srinivas Narayana, Ori Rottenstreich, S. Muthukrishnan, and Jennifer Rexford. Heavy-hitter detection entirely in the data plane. In *Proc. of ACM SOSR*, 2017.
- [70] Muhammad Tirmazi, Ran Ben Basat, Jiaqi Gao, and Minlan Yu. Cheeta: Accelerating database queries with switch pruning. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 2407–2422, 2020.
- [71] Amin Vahdat, Mohammad Al-Fares, and Alexander Loukissas. Scalable commodity data center network architecture, July 9 2013. US Patent 8,483,096.
- [72] Erico Vanini, Rong Pan, Mohammad Alizadeh, Parvin Taheri, and Tom Edsall. Let it flow: Resilient asymmetric load balancing with flowlet switching. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 407–420, 2017.
- [73] Qing Wang, Youyou Lu, and Jiwei Shu. Sherman: A write-optimized distributed b+tree index on disaggregated memory. In *Proceedings of the 2022 International Conference on Management of Data*, New York, NY, USA, 2022. Association for Computing Machinery.
- [74] Theophilus Wellem, Yu-Kuen Lai, Chao-Yuan Huang, and Wen-Yaw Chung. A flexible sketch-based network traffic monitoring infrastructure. *IEEE Access*, 7:92476–92498, 2019.
- [75] Jiarong Xing, Kuo-Feng Hsu, Yiming Qiu, Ziyang Yang, Hongyi Liu, and Ang Chen. Bedrock: Programmable network support for secure rdma systems. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 2585–2600, 2022.
- [76] Jiarong Xing, Wenqing Wu, and Ang Chen. Ripple: A programmable, decentralized link-flooding defense against adaptive adversaries. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 3865–3881, 2021.
- [77] Tong Yang, Jie Jiang, Peng Liu, Qun Huang, Junzhi Gong, Yang Zhou, Rui Miao, Xiaoming Li, and Steve Uhlig. Elastic sketch: Adaptive and fast network-wide measurements. In *Proc. of ACM SIGCOMM*, 2018.
- [78] Minlan Yu, Lavanya Jose, and Rui Miao. Software defined traffic measurement with opensketch. In *Proc. of USENIX NSDI*, 2013.
- [79] Matei Zaharia, Mosharaf Chowdhury, Michael J. Franklin, Scott Shenker, and Ion Stoica. Spark: Cluster computing with working sets. In *USENIX Workshop on Hot Cloud (HotCloud)*, 2010.
- [80] Yikai Zhao, Kaicheng Yang, Zirui Liu, Tong Yang, Li Chen, Shiyi Liu, Naiqian Zheng, Ruixin Wang, Hanbo Wu, Yi Wang, et al. Lightguardian: A full-visibility, lightweight, in-band telemetry system using sketchlets. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, pages 991–1010, 2021.